A Highly Available & Scalable Tiny SNS Design Document

Author: Diego Castellanos

Date: 3/11/2025

Project Overview

A Highly Available & Scalable (HAS) Tiny SNS builds on top of the architecture that allows clients to communicate with a system that is scaled horizontally. The objective of this refactor is to build a more robust system by introducing system design concepts availability and fault tolerance through techniques like mirroring and synchronization via asynchronous message brokers with RabbitMQ.

Requirements

The primary goal of this system redesign is to introduce high availability and scalability without impacting the user experience. All the changes made to this next iteration of Tiny SNS are completely transparent to the clients. In other words, clients interact with the system exactly as before, yet there are major architectural changes on the backend to support robustness and resilience.

Functional Requirements

- **Master-Slave Replication**: Messages posted to a master server must be reliably mirrored to a corresponding slave server for redundancy.
- **Asynchronous Synchronization**: Synchronizers must asynchronously publish and consume updates about client relationships and timelines using RabbitMQ.
- **Cluster Awareness**: Synchronizers must be aware of which clients belong to which clusters to route messages and updates correctly.
- **Coordinator Role Assignment**: The coordinator must be responsible for assigning server and synchronizer roles and providing this information to the rest of the system.

Non-Functional Requirements

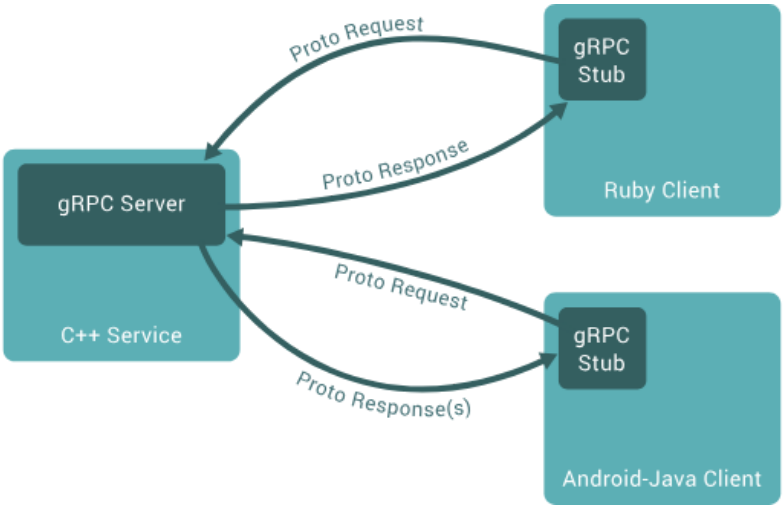
- **High Availability**: The system must tolerate individual server or synchronizer failures without impacting the user experience.

- **Transparency**: The upgraded architecture must not require changes in client logic or behavior.
- **Performance**: Timeline streaming and data synchronization must be efficient, minimizing latency and overhead.
- **Fault Detection:** Server failure must be detected via heartbeats, and slave promotion should occur automatically but lazily upon demand.
- **Extensibility**: The design should support future enhancements, such as proactive failover or stronger consistency guarantees.

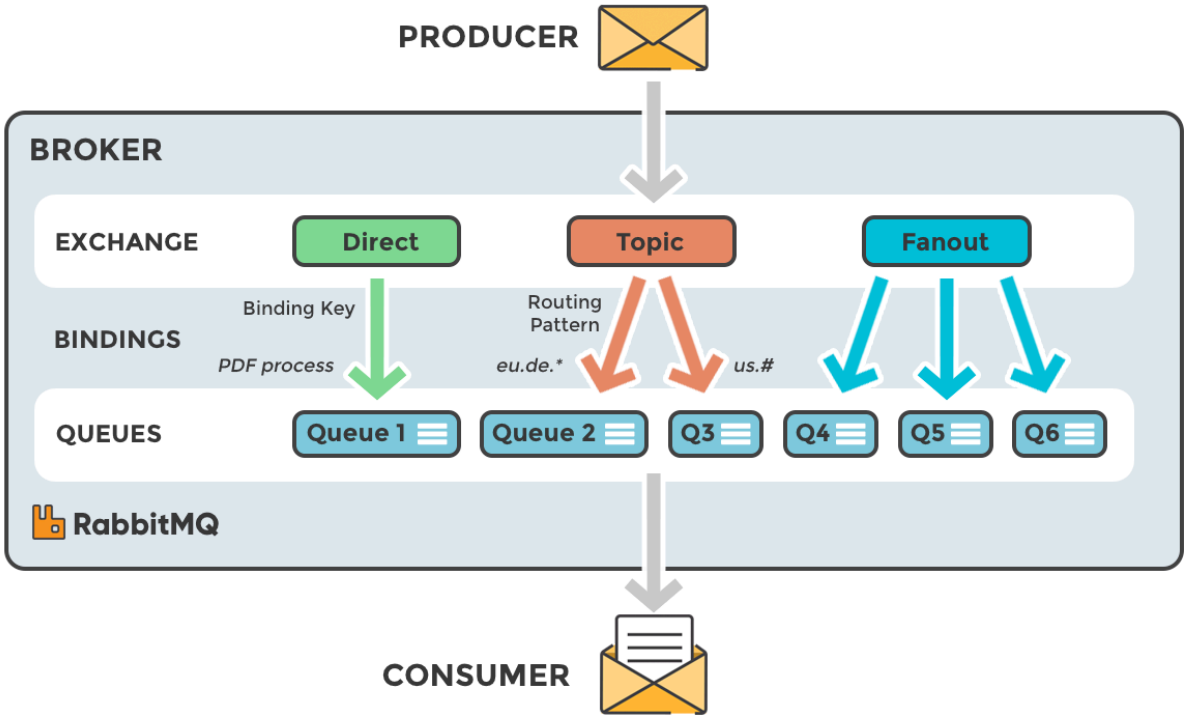
System Architecture

Communication

This project uses gRPC and Protocol Buffers due to their high performance in client-server communication and support for bidirectional streaming, which is essential for the TIMELINE feature. Communication between client-coordinator, client-server, server-coordinator, and synchronizer-coordinator all occur via gRPC.



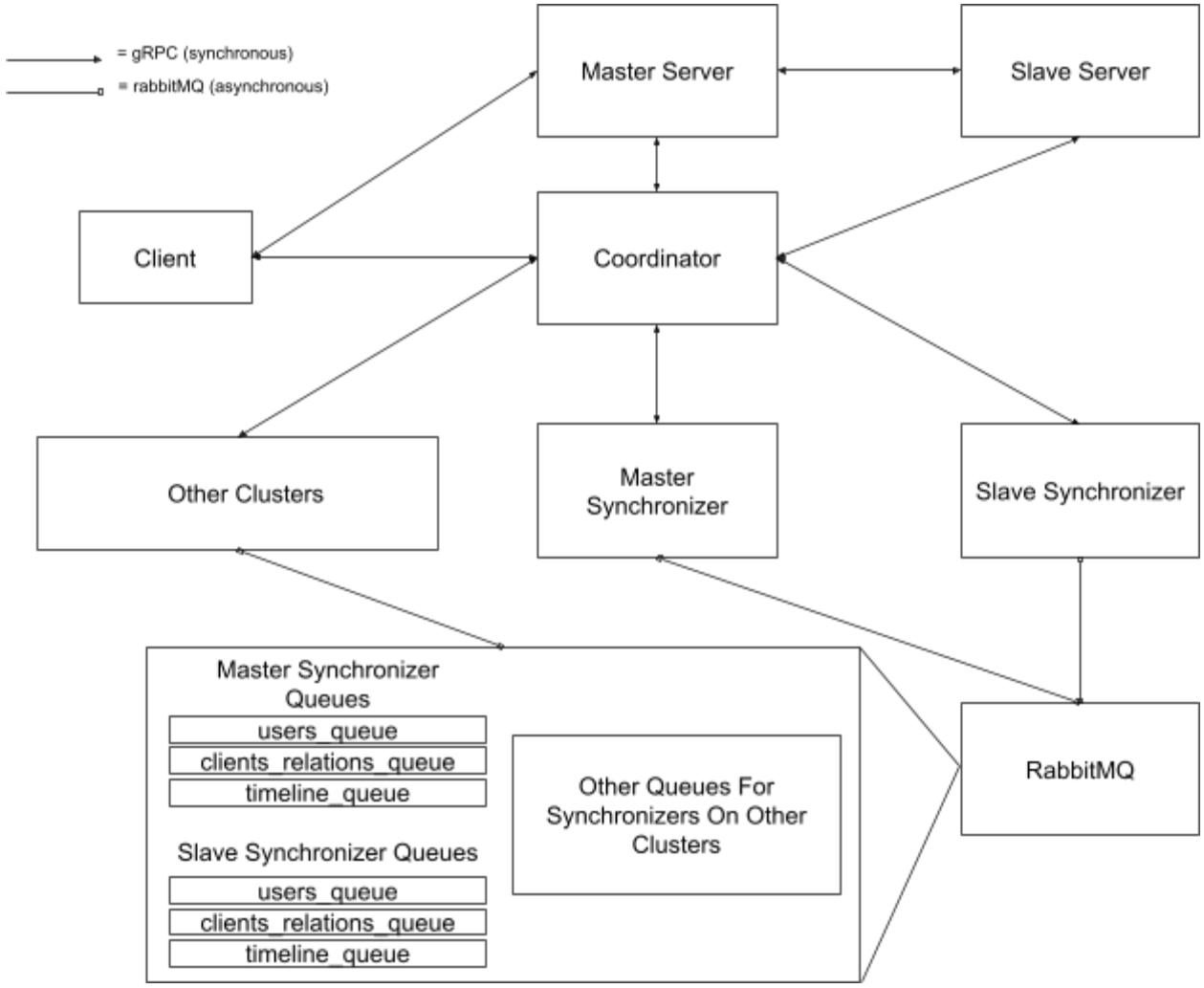
Another method of communication, RabbitMQ, an open-source message broker, is leveraged for asynchronous communication between follower synchronizers. Messages by follower synchronizers are published without blocking and are processed by other follower synchronizers whenever consuming from desired queues.



High-Level Design

There are a lot of moving parts to this system. Given that there is a lot of sharing of data via asynchronous communication, it can be difficult to pinpoint where data is stored and how it is shared. Below is a simplified design that showcases how all the different components of the system interact with each other.

.



Processes & Data Design

Coordinator

The coordinator serves the same purpose as before, but with additional responsibilities. Most of these responsibilities involve assigning roles to servers, providing synchronizers with information regarding available synchronizers across the system, and handling failover for server processes.

In my design, I found it necessary to store clients on the coordinator as a means to communicate with follower synchronizers what clients they are responsible for on their cluster. This allows synchronizers the opportunity to add clients to their all_users.txt file before consuming messages about the clients on other clusters.

```
rpc GetUsersOnCluster (ID) returns (AllUsers) {}
```

```
// create a vector of clients according to which cluster they are in
std::vector<std::unordered_set<std::string>> users(3);
```

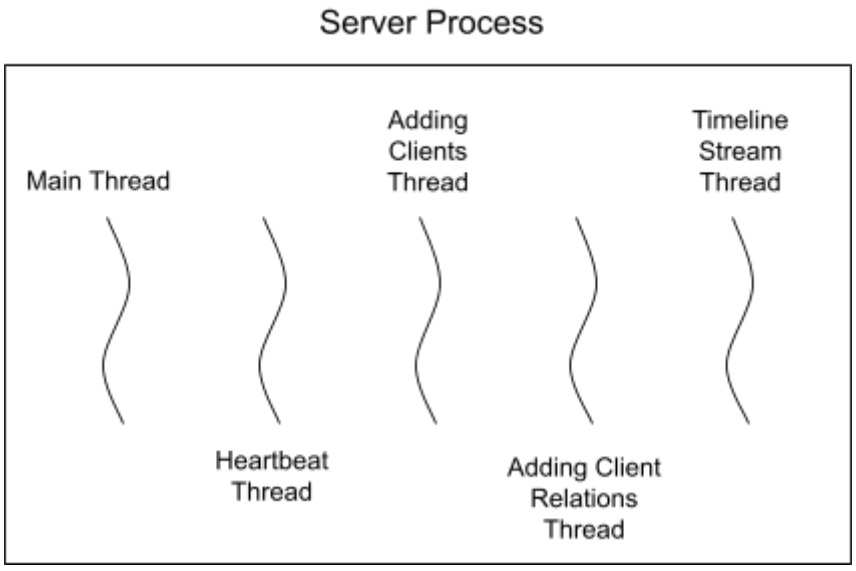
Servers

Servers play the same role as before, but to a much more complex degree to account for the changes in the system. Most notably, servers are categorized into ***master*** and ***slave*** servers, although they are not mutually exclusive as mentioned in the Coordinator explanation.

Master servers are tasked with communicating with clients, as before, but also with the ***slave*** server on their cluster as well. The latter mirrors everything done by the former. The system accomplishes this by having the ***master*** forward requests made by a client to the ***slave***. Replication is completed this way, leading to an overall more fault-tolerant design.

Servers are also responsible for updating timeline files for intra-cluster relationships (relationships between 2 clients on the same cluster as the server) and the streaming of posts. Only when relationships are inter-cluster (between 2 clients on different clusters) do follower synchronizers handle timeline files.

While the server could naively read from files using named semaphores whenever clients make requests, I decided to take a more extensible approach that also made use of caching data from disk to RAM to update the container storing clients. This behavior was achieved using threads which concurrently add clients to the container and update their following relationships intermittently (every 5 seconds). The server also streams posts to clients in timeline mode from followees in other clusters using a scheduled thread.



Follower Synchronizers

The 3 queues declared for asynchronous communication between the follower synchronizers were more than enough to share the state of a cluster with respect to the rest of the system. The publishing and consumption of timeline messages was done in a similar fashion to how it's done for clients and their relations. The posts made by a client are published only to the synchronizers responsible for the client's followers. Therefore, timeline files for clients are only present on the cluster the client is assigned to. An example of a JSON timeline message a *master* follower synchronizer might publish is shown below:

```
{
   "timeline": {
      "client_name": [["post1_timestamp", "post1_URL", "post1_content"],
                      ["post2_timestamp", "post2_URL", "post2_content"],
                      ...]
                }
}
```

Failure Handling

The logic behind fail-over is straightforward and implemented by the coordinator. At a high-level, whenever a server fails, the coordinator will record that it is no longer active after two missed heartbeats. It is only until a user sends a request to the coordinator to login to the Tiny SNS that the coordinator will promote the *slave* server and synchronizer to *masters*, and demote the *master* synchronizer. Therefore, the slave promotion is completed lazily.

Additional Notes

Thanks to Ivan Zaplatar, I was able to make changes to the RabbitMQ starting code for more robust consumption and publishing. I implemented his suggestion of utilizing the `amqp_envelope_t`'s routing key populated by `amqp_consume_message()` to route the message to the correct message handler.

Conclusion

The Highly Available & Scalable (HAS) Tiny SNS design enhances the reliability, fault tolerance, and scalability of the original system by introducing well-established distributed systems principles such as replication, asynchronous messaging, and lazy failover. Through the integration of gRPC and RabbitMQ, the system achieves a hybrid communication model that supports both real-time interactions and decoupled synchronization across clusters.

By introducing master-slave server roles and leveraging follower synchronizers with clearly defined responsibilities, this design enables horizontal scalability without compromising data consistency. The thoughtful use of threading, caching, and periodic updates allows for more efficient memory and resource management, especially under high load.

Overall, this refactored architecture lays the groundwork for a resilient and extensible social network system, capable of supporting a growing user base while maintaining responsiveness and consistency across distributed clusters.